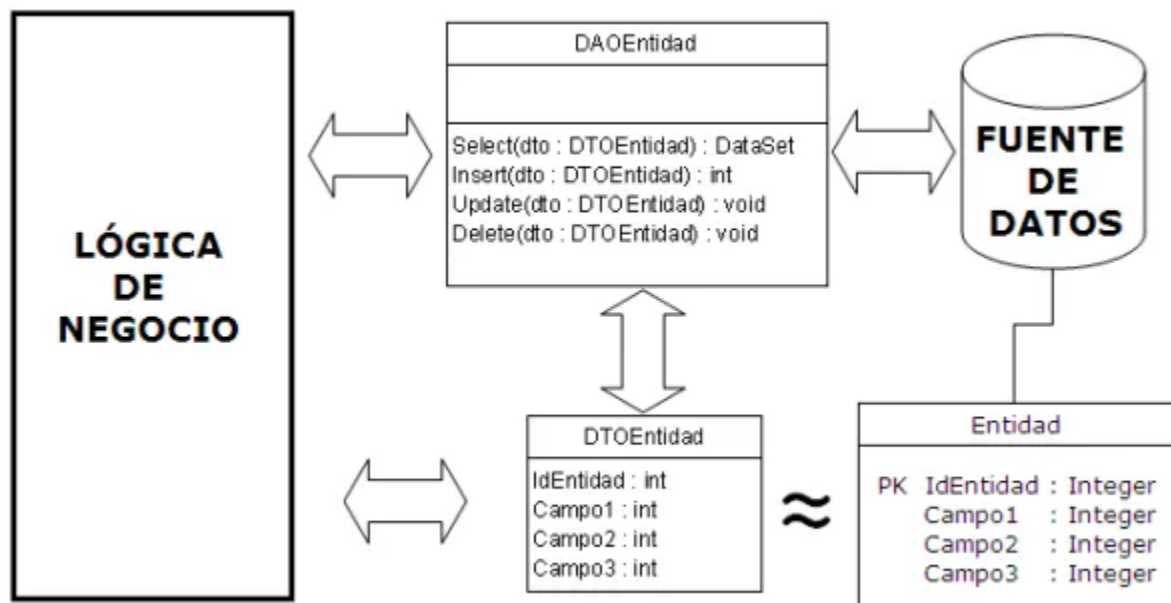


b)

PATRON DAO

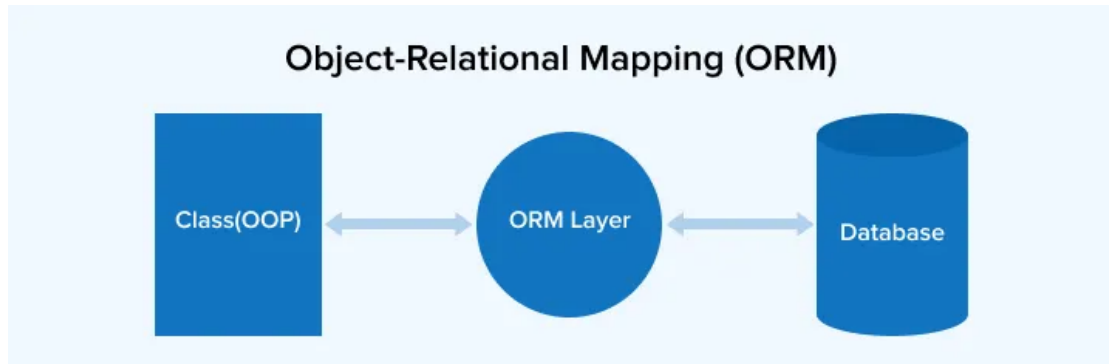
-Un patrón es un par problema/solución con nombre que se puede aplicar a nuevos contextos, con consejos acerca de cómo aplicarlo en nuevas situaciones y discusiones sobre sus compromisos. El patrón DAO es un patrón de diseño estructural que actúa como intermediario entre la lógica de negocio y la capa de persistencia de una aplicación, su objetivo principal es aislar y encapsular todas las operaciones de acceso a datos para que el resto de la aplicación pueda abstraerse de eso. Sirve para reutilizar código, facilita el testing, mantenimiento y desacoplamiento.



ORM

- Un ORM (Object-Relational Mapping) es una técnica de programación que actúa como puente para persistir en bases de datos relacionales objetos del paradigma orientado a objetos y abstraer el proceso de interactuar con estas mismas usando el mismo lenguaje de programación que el código fuente. Un ORM mapea los elementos de la base de datos con los elementos del código: Tabla -> Clase, Columna -> Atributo, Fila -> Objeto.
- Los lenguajes que pueden interactuar con un ORM son todos aquellos que fueron diseñados total o parcialmente bajo el POO, esto tiene sentido porque dado el caso de un lenguaje que no soporte la definición de clases y la instanciación de objetos entonces no habría necesidad de mapear elementos del código a elementos de base de datos.
- Los ORM más usados en la actualidad son SQLAlchemy(Python), Hibernate(Java), Gorm(Go), Prisma(Javascript/Typescript), Entity Framework Core(C#), Eloquent(PHP), Active Record(Ruby).

- Ventajas: Productividad a la hora de escribir menos código genérico, Abstracción de la base de datos, Seguridad para prevenir inyecciones SQL, Mantenibilidad al resultar en código más limpio y coherente.
- Desventajas: Las consultas SQL generadas por un ORM pueden ser menos eficientes, problema N+1 que es un error común de rendimiento donde el ORM realiza decenas o cientos de consultas a la base de datos de manera inadvertida dentro de un bucle en lugar de hacer una sola consulta optimizada.



PATRONES UTILIZADOS EN UN ORM

Patrón	Concepto y Propósito	Abstracción
Representación de Objetos como Tablas	Mapeo directo de estructura: Define la correspondencia básica entre el paradigma Orientado a Objetos y el esquema relacional.	• Propone la definición de una tabla en la base de datos relacional por cada clase de objeto persistente. • Los atributos de tipos primitivos (número, texto, booleano, etc.) se corresponden directamente con las columnas de la tabla. • Cada instancia u objeto guardado representa una fila o registro.
Identificador de Objeto (OID)	Identificación única y consistencia: Permite relacionar de forma unívoca objetos con registros y evita la duplicación de objetos al materializarlos	• Asigna un OID único (normalmente alfanumérico o numérico) a cada objeto/registro.

	repetidamente.	• Cada tabla utiliza este OID como su clave primaria y cada objeto de software encapsula directa o indirectamente dicho ID. • Garantiza que un registro de la BDR se corresponda con una única instancia en memoria.
Representación de las Relaciones de los Objetos en Tablas	Mapeo de asociaciones: Resuelve cómo trasladar las asociaciones de la Orientación a Objetos (uno-a-uno, uno-a-muchos, muchos-a-muchos) a un modelo relacional.	• Uno a Uno: Coloca una clave ajena (FK) de tipo OID en una/ambas tablas, o crea una tabla de asociación. • Uno a Muchos (Colecciones): Crea una tabla asociativa o auxiliar que registra los OIDs de los objetos conectados. • Muchos a Muchos: Utiliza una tabla asociativa/auxiliar que vincula los OIDs de ambas entidades.

JPA

Provee un estándar al desarrollador de aplicaciones con java que permite administrar y manipular datos almacenados en una base de datos relacional utilizando un modelo de dominio en Java. Nos permite establecer asociaciones entre clases de Java directamente con las tablas de la base de datos, de manera que los objetos que instanciamos se traducen directamente en registros de la BBDD; también brinda un lenguaje de consulta propio(JPQL) que es una variante orientada a objetos de SQL. Ofrece un Gestor de entidades que nos permite manejar el ciclo de vida de las entidades desde una interfaz centralizada.

En términos de optimización y rendimiento, JPA incorpora un sistema de almacenamiento en caché estructurado en dos niveles: Persistencia dentro de una misma transacción y a nivel de aplicación (datos entre distintas sesiones).

Una clase es de tipo entidad cuando representa un objeto de dominio persistente cuyo estado es administrado por la especificación de JPA y almacenado en una base de datos relacional. En la práctica, una entidad funciona como una representación orientada a objetos de una tabla de la base de datos, donde cada instancia u objeto de esa clase corresponde a un registro o fila en dicha tabla.

Las anotaciones nos permiten configurar metadatos generales sobre el mapeo, comportamiento, las consultas y el rendimiento. Algunas de las anotaciones que se pueden usar en una clase de tipo entidad:

@Entity, @Table, @SecondaryTable, @Inheritance, @IdClass, @NamedQuery, @SqlResultSetMapping, @Access.

Las anotaciones usadas para validar la información de los atributos se ejecutan antes de cualquier inserción, actualización o eliminación de registros de la bbdd. Algunas de las anotaciones usadas son: @NotNull, @NotEmpty, @NotBlank, @Min(value), @Max(value), @Email, @Size(min=,max=), @Future.

RELACIONES ENTRE CLASES

En JPA existen cuatro tipos principales de relaciones entre entidades:

@OneToOne: Un usuario tiene una única cuenta



@OneToMany: Una empresa puede tener varios empleados



@ManyToOne: Muchos libros pueden ser publicados por la misma editorial



@ManyToMany: Un estudiante puede estar inscripto en varias materias y una materia puede tener varios estudiantes inscriptos



FLUJO DE MAPEO: ENTIDAD -> TABLA/RELACIONES

1. Definición de la clase entidad

El desarrollador define el modelo orientado a objetos utilizando anotaciones de metadata de JPA para especificar la estructura y sus relaciones. Anotaciones usadas:

- **@Entity**: Al colocar esto arriba del nombre de la clase, le avisas al motor de JPA que esta clase representa un registro en la base de datos.
- **@Table** (Opcional): Por defecto, JPA creará una tabla con el mismo nombre que tu clase, pero si quieres que la tabla se llame diferente (ej. tb_usuarios), usas **@Table(name = "tb_usuarios")**

2. Identificación de la Clave Primaria (@Id)

Mediante la anotación **@Id** (sobre el atributo correspondiente). A menudo se acompaña con **@GeneratedValue** para decirle a la base de datos que genere ese ID automáticamente de forma autoincremental.

3. Mapeo de Atributos a Columnas (@Column)

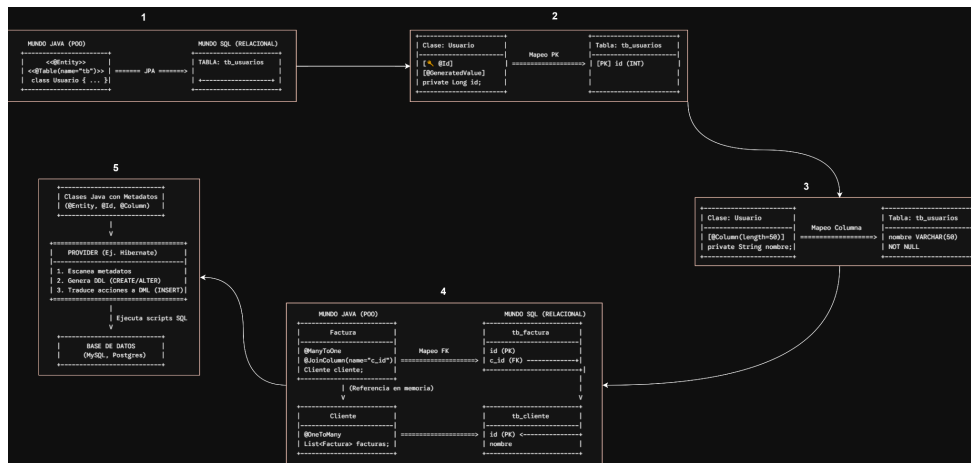
JPA mapea el resto de los atributos de tu clase a las columnas de la tabla (por ejemplo, un String pasa a ser un VARCHAR por defecto). Para personalizar este comportamiento (como no permitir valores nulos, cambiar longitudes o nombres), se utiliza la anotación **@Column**.

4. El Mapeo de las Relaciones (Las Claves Foráneas)

JPA traduce referencias a claves foráneas usando anotaciones de cardinalidad como **@OneToOne**, **@ManyToOne**, **@OneToMany** o **@ManyToMany**. Además, utiliza **@JoinColumn** del lado de la entidad correspondiente para crear físicamente esa clave foránea en la base de datos.

5. El Motor en Tiempo de Ejecución (El "Provider")

(Se necesita de un "Provider" como Hibernate) Al ejecutar la aplicación, este motor escanea las clases buscando **@Entity**, lee los metadatos de las anotaciones, genera internamente el código SQL puro (DDL) para crear o alterar las tablas en el motor de base de datos, y traduce las acciones de tus objetos a sentencias DML como INSERT INTO.



JPQL

Es un lenguaje de consultas orientado a objetos definido como parte de la especificación de JPA. Permite la realización de consultas sobre el modelo de clases de entidad de la aplicación en lugar de consultar directamente las tablas y columnas de la base de datos relacional.

Similitudes: Ambos comparten la misma estructura de cláusulas fundamentales, empleando palabras clave como SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY y JOIN. Asimismo, coinciden en el soporte para funciones de agregación como COUNT, SUM, AVG, MIN y MAX, y disponen de capacidades equivalentes para filtrar datos mediante condiciones lógicas.

Diferencias: SQL opera sobre tablas, filas y columnas de una base de datos relacional, mientras que JPQL trabaja exclusivamente sobre clases, objetos y atributos de Java. JPQL es case sensitive y SQL no lo es. En SQL es obligatorio declarar explícitamente las condiciones de unión mediante claves primarias y foráneas utilizando la sintaxis JOIN ON, mientras que en JPQL es posible navegar a través de los atributos de asociación previamente mapeados en las clases entidad. Adicionalmente, JPQL ofrece portabilidad garantizada entre distintos motores de base de datos.

Observacion	JPQL	SQL
Selección simple	SELECT e FROM Empleado e WHERE e.salario > 50000	SELECT e.id, e.nombre, e.salario FROM empleados e WHERE e.salario > 50000
Navegación a través de una	SELECT e	SELECT e.id, e.nombre, e.salario,

relación de muchos a uno	FROM Empleado e WHERE e.departamento.nombre = 'Sistemas'	e.departamento_id FROM empleados e INNER JOIN departamentos d ON e.departamento_id = d.id WHERE d.nombre = 'Sistemas'
Agrupación de datos y el uso de funciones de agregación	SELECT p.categoria, COUNT(p) FROM Producto p GROUP BY p.categoria	SELECT p.categoria_id, COUNT(*) FROM productos p GROUP BY p.categoria_id
Parámetros nombrados para aplicar un filtro por fecha. En JPQL los parámetros se identifican mediante dos puntos.	SELECT c FROM Cliente c WHERE c.fechaRegistro >= :fechaInicio	SELECT c.id, c.nombre, c.fecha_registro FROM clientes c WHERE c.fecha_registro >= ?
Actualización basada en la relación con otra entidad	UPDATE Empleado e SET e.salario = e.salario * 1.10 WHERE e.departamento.nombre = 'Ventas'	UPDATE empleados SET salario = salario * 1.10 WHERE departamento_id IN (SELECT id FROM departamentos WHERE nombre = 'Ventas')